概念：把轨迹编译成静态技能

30 秒速记：

- **问题：** 人工技能难以规模化；只凭模型参数知识写出的技能，又容易漏掉真实工具、文件格式和执行环境里的坑。
- **办法：** 冻结模型运行任务，把成功与失败轨迹先压缩成局部经验，再并行生成补丁、分层合并为一个技能目录。
- **产物：** 主要是 `SKILL.md + references/*.md`，不是新模型权重，也不是测试时检索的轨迹库。
- **关键边界：** MAP 通常看不到原始日志或 Excel，只读取前序分析器提炼出的 cause/memory records。

两种演化起点

概念	起点与目标
Deepening	从人工编写的强技能开始，补足真实执行才暴露的缺口
Creation	从 LLM 仅凭参数知识生成的弱草稿开始；并非空目录
Failure Cause	这次为什么错，强调根因及其与当前技能的关系
Memory	下次应遵守什么可迁移原则，是给 MAP 的压缩证据
Patch	对冻结技能提出的结构化编辑，不立即写入共享文件

论文写成“轨迹收集—并行补丁—分层整合”三步；公开代码进一步拆成：

```
执行并记日志 → 独立评分并标记成败
→ 失败/成功分析器生成 records
→ MAP 并行提补丁 → REDUCE 树状合并
→ TRANSLATION 校正编辑锚点 → Python APPLY → 格式验证
```

每个 MAP worker 都拿到同一份冻结技能 S_0，因此补丁不会依赖“谁先改了文件”。REDUCE 再统一去重、化解冲突，保留跨轨迹反复出现的标准操作规程。最终推理只加载整合后的技能，不需要参数更新或检索历史轨迹。

`cli_skill_preloaded_agent_105-24.md` 是什么？

它是实例 `105-24` 的 Markdown 版 ReAct 对话日志：包含预加载技能后的 system prompt、带路径和评分区域的任务、逐轮 assistant 响应与工具 Action、Observation，以及执行结束状态和轮数。`105-24` 在代码中只是完整 `instance_id`，不能拆解成“第 105 题第 24 小题”。它不内嵌工作簿；`input.xlsx`、`output.xlsx` 和中间脚本保存在独立工作目录。后置评测后，日志才会被改名为 `_SUCCEED.md` 或 `_FAILED.md`。

流程：一条失败如何变成规则

下面是教学示例，不是论文中的原始样本。任务要求把公式写到 `Summary!K6:K20`，智能体却使用 `wb.active`，误写进当前活动表 `Data`。

1. **执行。** `cli_skill_preloaded_agent` 把技能全文装入 system prompt，只给智能体 bash 工具；ReAct 循环检查文件、写脚本并生成 `output.xlsx`。
2. **先判"完成"，再判"正确"。** runner 看到输出文件就把本次执行记为成功；这只表示产物存在。后置 `evaluate_with_official.py` 才对照 gold 检查 `answer_position`：Soft 是通过 testcase 的比例，Hard 只有全部通过才为 1，实例的正确性标签取 Hard。
3. **划分成败。** `analyze_results.py` 按 instance id 匹配评分 JSON 和日志，再依据 evaluator 的 `success` 改名为 `_SUCCEED` 或 `_FAILED`。因此日志内部的 `RESULT Success` 与文件名里的"答对"不是同一件事。
4. **调查失败。** 错误分析器获得完整日志，以及隔离目录中的 `input.xlsx`、`output.xlsx`、`gold.xlsx` 和中间脚本。它可用 bash 检查 sheet、公式和值，定位到活动工作表选错。
5. **验证修复。** 分析器把实现改成 `wb["Summary"]`，生成候选输出，再调用 `evaluate_output`。只有至少一次出现 PASS、写出 `evaluate_passed.flag` 的目录式报告，才会进入后续 error records。
6. **压缩证据。** 调查被改写成 Failure Cause："依赖活动表导致写错位置"，以及 Failure Memory："按名称显式选择工作表，提交前回读评分区域"。无法形成可验证修复的失败会被过滤。
7. **成功侧更轻。** 成功分析器只读取已标为 `_SUCCEED` 的日志，单次调用 LLM 提取 Success Memory；它不重新打开工作簿，也不验证"某个动作是否真是成功原因"。
8. **生成与合并。** MAP 看到 S_0、一级 `references/` 和结构化 records；Combined 模式同时保留 failure cause、failure memory、success memory。多个 batch 并行产生 patch，REDUCE 分层合并，TRANSLATION 把模糊章节名或文本锚点校正为精确编辑，最后由 Python 写入文件。

这里有一个有意设计的信息瓶颈：

```
日志 + input/output/gold + 修复过程
                ↓
      Cause / Memory records
                ↓
          Skill Patch
```

好处是降低上下文成本，防止 MAP 被个案细节淹没；代价是分析器若压缩错了，MAP 无法回到原始证据复核。修复 PASS 也不是形式化因果证明：公开回退评测主要比较评分区域的单元格值，不保证公式文本、样式、评分区外副作用或报告里的根因解释都正确。

判断：实验证据与实现边界

论文做了哪些实验？

范围	设置与结论
SpreadsheetBench	400 个 Verified 样本按 200 演化／200 留出划分，另测排除演化样本后的全量任务；结果取 3 个随机种子平均
OOD 表格问答	把 WikiTableQuestions、HiTab 转成表格任务，检验技能是否只记住演化题
跨模型	35B／122B 互为 skill author 与 user，并测试 Gemma-4、GPT-5.5-high 跨家族迁移
数学	400 个 DAPO 演化、100 个留出 DAPO 和 AIME 2026；+Error 最稳定，部分 Combined 指标会退化
DocVQA	50 个样本演化，5,299 个留出评测；所有演化技能都优于 No Skill
更广文件任务	PDF `76.9→85.3`、PPTX `72.5→88.8`、DOCX `79.7→87.5`，均从源任务迁移到独立目标

最值得记住的数字

比较	论文报告结果
122B Human-Written → Deepening +Combined	Vrf `48.33→69.83`，Soft `36.30→47.17`，Hard `17.03→29.53`
35B 编写 Creation +Error → 122B 跑 WikiTQ	`23.73→81.38`，提升 `57.65` 个百分点
ReasoningBank → +Combined，122B	Vrf `56.00→69.83`，Hard `21.30→29.53`
顺序编辑 → 并行整合	技能生成约 `60/15 分钟→3 分钟`；不是单题推理延迟
DocVQA，122B +Combined	最终 ANLS `0.8833`、Accuracy `92.52`；相对 No Skill 分别 `+0.2534`、`+22.25` 个百分点

这些结果支持三点：整合成一个常驻技能通常优于测试时 top-1 检索孤立记忆；技能能跨模型规模、家族和 OOD 任务迁移；能检查真实产物并验证修复的 agentic error analyst，通常比只读日志的一次 LLM 调用更可靠。但补丁会互相增益或干扰，收益不是逐条可加，也不保证每个指标都提升。

代码审查后的保留意见

- 固定提交 `3d0b52a` 只公开电子表格流水线；数学、DocVQA、PDF、PPTX、DOCX 的实验实现未发布。
- 仓库缺少 CLI 强制要求的 `skills/skill-creator/scripts/quick_validate.py`，README 的非 dry-run 演化命令在纯净 checkout 中不能原样跑完。
- `evaluation_official` 无法导入时会直接退回本地 openpyxl comparator；应称“official-compatible wrapper”，不能假定总在调用官方 scorer。
- 并行 patch 路径的安全校验弱于顺序实现；写盘后的最终验证失败没有自动回滚，只有 CLI 预先留下的备份可供人工恢复。
- 本报告做了固定版本静态审查、Python 语法与译稿编译检查，未配置论文模型、完整 API 和全部基准，不能视为独立复现实验。

来源与许可： 论文 arXiv v5；代码固定提交。数字来自论文，未独立复现。译稿是 CC BY 4.0 非官方翻译，图内文字未译；下载包已剔除单独许可的 TeX 构建文件，完整上传包见 arXiv source。代码仓库许可并不统一：README 声明 Apache-2.0，部分 released skills 另有限制，故这里只链接固定提交。